Unified Digital Wellness System

A PROJECT REPORT

Submitted by

Abhijeet Sandilya (24MIM10074)

Ayush Chandravanshi (24MIM10221)

Atharva Padole (24MIM10038)

Soma Vashista (24MIM10098)

in partial fulfillment for the award of the degree

of

INTEGRATED MASTER OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE

SCHOOL OF COMPUTING SCIENCE ENGINEERING AND ARTIFICIAL INTELLIGENCE (SCAI)

VIT BHOPAL UNIVERSITY

KOTHRIKALAN, SEHORE
MADHYA PRADESH – 466114

APRIL , 2026

VIT BHOPAL UNIVERSITY, KOTHRIKALAN,SEHORE

MADHYA PRADESH – 466114

BONAFIDE CERTIFICATE

This is to certify that the thesis titled AI-Driven Student Well-Being Monitoring System, submitted by Abhijeet Sandilya (24MIM10074), Ayush Chandravanshi (24MIM10221) , Atharva Padole (24MIM10038), and Soma Vashista (24MIM10098) to VIT Bhopal University, in partial fulfillment of the requirements for the award of the degree of **Integrated Master of Technology in Artificial Intelligence**, is a bona fide work carried out under my supervision. The thesis fulfills the requirements as per the regulations of this University and, in my opinion, meets the necessary standards for submission. The contents of this thesis have not been submitted, and will not be submitted, either in part or in full, for the award of any other degree or diploma, and the same is certified.

Supervisor

Signature
Name: Dr Jyoti Chauhan
Designation: Assistant Professor
Date:

Program Chair

Signature
Name: Dr.S Devaraju
Designation: Assistant Professor
Date

Reviewer 1

Signature
Name: Dr.S.Periyanayagi
Designation: Assistant Professor
Date:

Reviewer 2

Signature
Name: Dr.Abha Trivedi
Designation: Assistant Professor
Date

A viva-voce was contucted on

ACKNOWLEDGEMENT

I am deeply indebted to **Dr. G. Viswanathan**, Chancellor of VIT Bhopal University, and **Dr. T.B. Sridharan**, Pro Vice-Chancellor, VIT Bhopal University, for their unwavering support and guidance. Their leadership and vision have been a constant source of inspiration throughout my academic journey. Express my profound gratitude to **Dr. Pon Harshavardhan**. Dean of SCAl, for his encouragement and invaluable support at every stage, which significantly contributed to the successful completion of my project and degree. My heartfelt thanks go to **Dr. S. Devaraju**, Program Chair of Integrated M.TechArtificial Intelligence, and **Dr. Venkat Prasad Padhy**, Division Head of the Al Division, for their timely assistance, guidance, and invaluable insights during the course of my research.

A special note of appreciation reserved for my supervisor, **Dr Jyoti Chauhan**, whose constant support, patience, and expertise were instrumental in completing this project. Her guidance, encouragement, and clarity in addressing my queries made this journey both meaningful and enriching.

I would like to express my sincere gratitude to **Dr.S.Periyanayagi** and **Dr.Abha Trivedi**. for their valuable feedback and insightful reviews.Lastly, I extend my deepest gratitude to my parents and friends, whose unwavering encouragement and support have been my driving force throughout this research endeavor. Their belief in me has been a source of strength and motivation.

Abbreviations

AI	Artificial Intelligence
CNN	Convolutional Neural Network
CV	Computer Vision
DWS	Digital Wellness System
EAR	Eye Aspect Ratio
FPS	Frames Per Second
GUI	Graphical User Interface
HTTP	HyperText Transfer Protocol
MAR	Mouth Aspect Ratio
MTV	Model-Template-View
MVC	Model-View-Controller
ORM	Object-Relational Mapper
TTS	Text-to-Speech
UI	User Interface
URL	Uniform Resource Locator
WSGI	Web Server Gateway Interface

Figures and Graphs

Tables

Abstract

This project presents an AI-Driven Digital Wellness Monitoring System—a web-based platform that leverages computer vision and machine learning techniques to monitor the physiological and behavioural wellness indicators of users in real time. The system uses Google's MediaPipe Face Landmarker model to detect and track 478 three-dimensional facial landmarks from a standard webcam feed, enabling the calculation of clinically grounded wellness metrics without any specialized hardware.

The core analytical engine computes the Eye Aspect Ratio (EAR) to detect prolonged eye closure indicative of fatigue or drowsiness, the Mouth Aspect Ratio (MAR) to identify frequent yawning as a marker of physical tiredness, and a Blink Rate tracker to assess visual strain and attentiveness. When monitored values deviate from established healthy thresholds, the system triggers real-time audio alerts using the pyttsx3 Text-to-Speech library, providing immediate, non-intrusive feedback to the user.

The entire application is engineered using the Django web framework following the Model-Template-View (MTV) architectural pattern, with SQLite as the backend database for persisting historical health records. OpenCV handles all video stream acquisition and frame processing tasks. The resulting platform offers an accessible, locally deployable, and privacy-preserving solution for continuous digital wellness monitoring, with direct applications in student fatigue detection, workplace ergonomics, driver safety monitoring, and online learning engagement analytics.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	List of Abbreviations List of Figures and Graphs List of Tables Abstract	

CHAPTER-1

PROJECT DESCRIPTION AND OUTLINE

1.1 Introduction

The modern world is witnessing an unprecedented convergence of digital life and human health, a phenomenon that is fundamentally reshaping the landscape of personal well-being. As individuals—from young students to working professionals—spend increasingly long hours in front of screens for study, work, and leisure, the human body and mind are being subjected to a sustained physiological strain for which they were not evolutionarily designed. This shift has transformed what were once occasional discomforts into a critical public health concern, with the cumulative toll manifesting in a range of debilitating conditions. Issues such as digital eye strain, characterized by dryness, irritation, and blurred vision; cognitive fatigue, which impairs memory and decision-making; attention deficit, marked by an inability to maintain focus; and persistent drowsiness, a precursor to microsleep episodes, are now recognized as widespread occupational and academic hazards. Critically, conventional monitoring approaches—which rely on self-reporting, periodic health check-ups, or simple timer-based reminders—are ill-equipped to address these issues proactively, as they fail to capture the real-time, subtle physiological cues that precede a decline in wellness.

Within the specific context of intensive academic environments, such as educational institutions like VIT Bhopal University, this challenge is particularly acute. Students in such settings routinely engage in extended sessions of computer-based study, high-stakes online examination preparation, and intensive digital project work. This sustained and often uninterrupted screen exposure, frequently occurring without adequate breaks or conscious awareness of one's own physiological cues, creates a perfect storm for wellness degradation. The consequences are tangible and severe, leading to significant drops in academic performance due to impaired concentration, an increase in error rates during critical tasks, and the insidious development of long-term vision problems and musculoskeletal health issues such as chronic neck and back pain. The absence of an objective, automated system capable of continuously monitoring these wellness indicators

in real time represents a critical gap in the modern digital learning environment, leaving students vulnerable to preventable declines in both health and scholastic achievement.

This project directly addresses that gap by proposing an **AI-Driven Digital Wellness Monitoring System**—an intelligent, camera-based web application designed to operate non-intrusively. By utilizing a standard webcam, the system discreetly observes a user's facial behaviour and employs advanced computer vision and machine learning techniques to derive objective, actionable wellness metrics in real time. The system's architecture is built upon three integrated technological pillars that work in concert to transform raw video data into meaningful health insights.

The first pillar is real-time facial landmark detection, powered by Google's state-of-the-art MediaPipe Face Landmarker model. This sophisticated model precisely maps 478 three-dimensional facial keypoints from every frame captured by the webcam. These points are not merely two-dimensional coordinates; they represent a detailed, dynamic 3D mesh of the face, providing the high-fidelity spatial data necessary for accurate analysis. These landmarks serve as the raw data foundation from which all subsequent wellness metrics are derived, forming the sensory backbone of the entire system.

The second pillar is a dedicated wellness analytics engine that processes the stream of landmark coordinates to compute critical physiological indicators. This engine employs proven computational methods to calculate the Eye Aspect Ratio (EAR) , a precise metric used for drowsiness detection and comprehensive blink rate analysis. By continuously monitoring the EAR, the system can identify patterns indicative of fatigue and the early onset of sleepiness. Concurrently, the engine calculates the Mouth Aspect Ratio (MAR) , a key metric for yawning detection. Yawning is a strong, often unconscious, behavioral indicator of fatigue, and its automated detection adds a crucial second dimension to the system's assessment of user alertness.

The third pillar is a robust and user-centric web application framework built with Django. It provides structured data persistence via SQLite, allowing for the secure storage and longitudinal tracking of user wellness trends, which is essential for identifying chronic issues. Furthermore, it delivers a clean user interface that presents wellness metrics in an intuitive format, empowering users with self-awareness. Finally, it orchestrates a real-time

alert system powered by the pyttsx3 Text-to-Speech engine, which enables the application to provide immediate, non-visual auditory feedback. This allows the system to intervene at the moment of need—for instance, prompting a user to take a break upon detecting signs of significant fatigue—thereby fulfilling its core mission of proactive digital wellness management.

1.2 Motivation for the Work

The motivation for this project stems from the observable and documented rise in screen-time-related fatigue and the absence of affordable, accessible tools to detect and respond to it in real time. Traditional wellness interventions are reactive—a student only seeks help after symptoms like headaches, blurry vision, or extreme tiredness have already manifested. A proactive, automated system that detects the precursors of these conditions—prolonged eye closure, reduced blink frequency, frequent yawning—and immediately alerts the user represents a fundamentally superior approach to digital wellness management.

Furthermore, the availability of powerful, open-source computer vision libraries such as MediaPipe and OpenCV, combined with the maturity of the Django web framework, makes the implementation of such a system both technically feasible and academically novel. The project also aligns with the growing body of research in affective computing, which seeks to create systems that are sensitive to the human user's physiological and emotional state.

1.3 Introduction to Techniques Used

This project employs a diverse set of advanced technologies working in concert:

- MediaPipe Face Landmarker: Google's state-of-the-art face mesh solution, which detects 478 3D facial landmarks in real time from a single camera without any depth sensor.
- Eye Aspect Ratio (EAR): A geometric metric computed from six eye-landmark coordinates that quantifies the degree of eye opening. A sustained drop below a threshold value signals drowsiness.
- Mouth Aspect Ratio (MAR): An analogous geometric ratio computed from lip landmark coordinates to detect the wide mouth opening characteristic of a yawn.

- Blink Rate Tracking: The system counts blink events per unit time, with an abnormally low blink rate serving as an indicator of visual strain and reduced attentiveness.
- Django Web Framework: A Python-based, MTV-architecture web framework that provides ORM-based SQLite data persistence, URL routing, template rendering, and session management.
- OpenCV: The industry-standard computer vision library used for webcam stream capture, frame decoding, and image preprocessing.
- pyttsx3 (Text-to-Speech): A cross-platform Python TTS library used to deliver spoken wellness alerts without requiring an internet connection.

1.4 Problem Statement

Existing digital wellness solutions either rely on expensive proprietary hardware, require continuous active user input, or function only as passive reminder timers that do not adapt to the actual physiological state of the user. There is a critical need for a system that can autonomously, continuously, and unobtrusively monitor objective physiological cues of digital fatigue and visual strain, process them in real time using machine learning, and respond with immediate, personalized alerts—all within a lightweight, locally deployable web application that does not compromise user privacy.

1.5 Objectives of the Work

The primary objectives of this project are:

- To develop a real-time webcam-based facial monitoring system using the MediaPipe Face Landmarker model capable of tracking 478 facial landmarks.
- To implement physiological wellness metrics—EAR, MAR, and Blink Rate—derived from facial landmarks to detect drowsiness, fatigue, and visual strain.
- To trigger immediate, non-intrusive audio alerts via pyttsx3 TTS when monitored metrics deviate from established healthy thresholds.
- To build a full-stack web application using Django and SQLite to persist health session data, providing users with a historical view of their wellness trends.

- To deliver a locally deployable, privacy-preserving solution that requires only a standard webcam and a personal computer.

1.6 Organization of the Project

This report is organized into seven chapters. Chapter 2 reviews related literature on drowsiness detection, fatigue monitoring, and computer vision in health applications. Chapter 3 details the hardware and software requirements. Chapter 4 describes the system design, architecture, and module interactions. Chapter 5 provides a comprehensive technical implementation narrative. Chapter 6 discusses outcomes and real-world applicability. Chapter 7 concludes the report with limitations, future enhancements, and individual contributions.

1.7 Summary

This chapter has established the context, motivation, and objectives of the AI-Driven Digital Wellness Monitoring System. The project proposes a novel, integrated solution leveraging MediaPipe, OpenCV, Django, and pyttsx3 to provide real-time, objective monitoring of digital fatigue in a privacy-preserving, locally deployable web application.

CHAPTER-2

RELATED WORK INVESTIGATION

2.1 Introduction

The field of automated wellness and fatigue monitoring using computer vision has a rich body of research. This chapter reviews key prior work in three areas: real-time facial landmark detection systems, EAR/MAR-based drowsiness detection, and web-based health monitoring applications. The review establishes the context for this project's unique contributions.

2.2 Existing Approaches and Methods

2.2.1 Real-Time Facial Landmark Detection

Early facial landmark detection relied on Active Appearance Models (AAMs) and Constrained Local Models (CLMs), which were computationally expensive and brittle under real-world conditions such as varying illumination and partial occlusion. The introduction of deep learning-based methods dramatically improved robustness. The work of Kazemi and Sullivan (2014) demonstrated that ensemble regression trees could regress 68 facial landmarks in milliseconds, making real-time applications feasible. Google's MediaPipe Face Mesh and the subsequent Face Landmarker solution extended this to 468 and 478 landmarks respectively, predicting three-dimensional coordinates from a single RGB camera feed without a depth sensor, a significant advancement over earlier RGB-D approaches.

2.2.2 EAR-Based Drowsiness and Blink Detection

Soukupova and Cech (2016) introduced the Eye Aspect Ratio as a simple, elegant metric for real-time blink detection, demonstrating that a single threshold on this ratio reliably distinguishes open from closed eyes across diverse individuals. Subsequent research by Maggioni et al. and others extended this approach to driver drowsiness detection systems integrated into vehicles. However, most implementations relied on the dlib 68-landmark predictor, which, while effective, is less accurate and computationally heavier than modern MediaPipe-based approaches. This project improves upon those baselines by employing MediaPipe's 478-landmark model for higher precision landmark localization.

2.2.3 Yawning Detection Using Mouth Metrics

Yawning has long been recognized as a reliable indicator of physiological fatigue. Studies using the Mouth Aspect Ratio—the ratio of mouth height to width, computed from lip landmarks—have demonstrated effective yawning detection rates above 85% in controlled settings. Existing systems typically treat yawning detection in isolation, whereas this project integrates it with blink rate analysis and EAR monitoring to provide a composite wellness score, offering a more holistic picture of the user's fatigue state.

2.2.4 Web-Based Health Monitoring Applications

The intersection of web development and real-time computer vision has been explored in projects such as online proctoring systems and telehealth monitoring dashboards. Flask and Django have been used as backends for serving processed video feeds and storing health logs. However, most existing academic implementations lack persistent data storage, historical trend analysis, or audio feedback mechanisms, limiting their practical utility as a standalone wellness tool.

2.3 Pros and Cons of Existing Approaches

The following table summarises the strengths and weaknesses of related approaches:

Approach	Pros	Cons
dlib 68-Landmark + EAR	Simple, well-documented	Slower, lower accuracy, 2D only
Active Appearance Models	Model-based, interpretable	Fragile under real-world conditions
MediaPipe Face Mesh (468 pts)	Fast, accurate, cross-platform	Limited to face region
MediaPipe Face Landmarker (478 pts)	State-of-the-art accuracy, 3D, blendshapes	Requires calibration for thresholds
Standalone scripts (no web framework)	Simple deployment	No data persistence or history

2.4 Issues and Observations from Investigation

The literature review reveals several gaps that this project addresses. First, no existing open-source system comprehensively integrates EAR-based drowsiness detection, MAR-based yawning analysis, and blink rate tracking within a single unified Django web application with persistent SQLite storage. Second, most drowsiness detection systems are designed for vehicular or clinical settings and are not adapted for the student/office digital wellness context. Third, privacy is rarely addressed—cloud-based processing of live webcam feeds raises significant data sovereignty concerns. The locally deployable, Django-based architecture of this project resolves all three issues.

2.5 Summary

This chapter has surveyed the landscape of facial landmark detection, EAR/MAR-based fatigue monitoring, and web-based health systems. The review validates the methodological choices made in this project—particularly the selection of MediaPipe Face Landmarker, the composite metric approach, and the Django framework—while identifying clear novelty in their integrated application to digital wellness monitoring.

CHAPTER-3

REQUIREMENT ARTIFACTS

3.1 Introduction

This chapter documents the hardware, software, data, and functional requirements necessary for the development and deployment of the Digital Wellness Monitoring System.

3.2 Hardware and Software Requirements

Hardware Requirements:

Component	Specification
Processor	Intel Core i5 (8th Gen or later) or equivalent AMD Ryzen 5
RAM	8 GB minimum (16 GB recommended for real-time video processing)
Storage	256 GB SSD (for fast read/write of SQLite DB and session logs)

Webcam	720p HD webcam (1080p preferred for improved landmark accuracy)
Display	1366×768 minimum resolution
Operating System	Windows 10/11, macOS 11+, or Ubuntu 20.04+

Software Requirements:

Component	Version / Type
Python	3.8 or later
Django	4.2.7 (Web Framework — MTV Architecture)
OpenCV	opencv-python 4.8.1.78 (Video Capture and Frame Processing)
MediaPipe	>=0.10.13 (Face Landmarker — 478 3D Landmarks)
pyttsx3	2.90 (Cross-Platform Text-to-Speech Engine)
NumPy	1.24.3 (Numerical Computations — EAR/MAR)
Pillow	10.1.0 (Image Handling and Processing)
SQLite	Built-in (Django ORM — Health Data Persistence)
Browser	Google Chrome, Mozilla Firefox, or Microsoft Edge

3.3 Specific Project Requirements

3.3.1 Data Requirements

The system does not require any pre-compiled external dataset for operation. The MediaPipe Face Landmarker employs a pre-trained model file (face_landmarker.task) distributed with the library. Health session data—including timestamped EAR values, MAR values, blink counts, and alert events—is stored in a locally managed SQLite database using Django's ORM. No facial images or raw video frames are ever persisted to disk, ensuring user privacy.

3.3.2 Functional Requirements

The functional requirements of the system include the following capabilities:

- Real-time webcam stream acquisition and frame-by-frame processing using OpenCV.
- Detection and tracking of 478 3D facial landmarks per frame using the MediaPipe Face Landmarker model.
- Computation of EAR from left and right eye landmark coordinates with per-frame drowsiness classification.
- Computation of MAR from upper and lower lip landmark coordinates with yawning event detection.
- Blink event detection through EAR threshold crossing and per-minute blink rate calculation.
- Real-time audio alert generation via pyttsx3 when EAR, MAR, or blink rate thresholds are breached.
- Persistent storage of wellness session logs in SQLite via Django ORM.
- User-facing dashboard displaying live metric values and historical trend charts.

3.3.3 Performance and Security Requirements

The system must process webcam frames at a minimum of 15 FPS to ensure meaningful real-time monitoring; MediaPipe's lightweight model architecture achieves this on standard hardware. All video processing occurs locally on the user's device—no frames or biometric data are transmitted to any external server, guaranteeing full data privacy.

The Django application enforces session-based authentication for the monitoring dashboard. The SQLite database stores only aggregated numeric metrics, never raw video or image data.

3.3.4 Look and Feel Requirements

The Django-rendered web interface must present a clean, professional dashboard with a live video feed panel displaying the current frame with landmark overlays, real-time numeric displays of EAR, MAR, and blink rate, colour-coded status indicators (green/yellow/red) corresponding to wellness states, and a historical data section presenting trend charts for session review.

3.4 Summary

This chapter has specified the complete hardware, software, functional, and non-functional requirements of the system. The lightweight dependency stack—Django, OpenCV, MediaPipe, pyttsx3, and NumPy—ensures the system is deployable on consumer-grade hardware without any cloud infrastructure.

CHAPTER-4

DESIGN METHODOLOGY AND ITS NOVELTY

4.1 Methodology and Objective

The system follows a modular design paradigm where each functional concern—video acquisition, landmark detection, wellness metric computation, alert generation, data persistence, and UI rendering—is encapsulated in a distinct module. This separation of concerns ensures testability, maintainability, and extensibility. The novelty of the design lies in the integration of Google's MediaPipe Face Landmarker (a state-of-the-art 3D landmark detection pipeline) with a full Django web application, enabling persistent health analytics capabilities that no comparable open-source tool currently provides.

4.2 Functional Module Design and Analysis

The system is composed of the following key functional modules:

- Video Acquisition Module: Interfaces with the webcam via OpenCV's VideoCapture API, decoding each JPEG frame into a NumPy array suitable for MediaPipe processing.
- Face Landmarker Module: Wraps the MediaPipe FaceLandmarker task configured with the face_landmarker.task model bundle. For each input frame, it returns a list of NormalizedLandmark objects (478 per detected face) containing normalised x, y, z coordinates.
- Wellness Metrics Module: Implements EAR computation using the six-landmark eye geometry formula, MAR computation from upper and lower lip landmarks, and a stateful blink counter that detects EAR threshold crossings.
- Alert Engine Module: A threshold-monitoring engine that evaluates metric values against configurable limits and invokes pyttsx3's speech synthesis for audio alerts, with a cooldown timer to prevent alert flooding.
- Data Persistence Module: Django ORM models (HealthSession, MetricSnapshot) mapped to SQLite tables via managed migrations. Each monitoring session creates a HealthSession record, and periodic MetricSnapshot records capture the instantaneous metric state.
- Web Interface Module: Django views render Jinja2/DTL templates for the login page, dashboard, and historical records pages. The live video feed is streamed as a multipart/x-mixed-replace HTTP response from a dedicated streaming view.

4.3 Software Architectural Design

The overall software architecture follows Django's Model-Template-View (MTV) pattern, which is semantically equivalent to the classical MVC pattern. The Model layer defines the database schema via Django ORM classes and manages all data access logic. The Template layer contains the HTML templates responsible for rendering the user interface, including the live video panel and metric dashboards. The View layer contains the Python

functions/classes that process HTTP requests, invoke the computer vision pipeline, and return HTTP responses.

The computer vision pipeline operates as a background processing thread that continuously reads frames from OpenCV, processes them through MediaPipe, computes wellness metrics, updates an in-memory state object, and streams annotated frames to the Django streaming view via a shared queue. This threading architecture decouples the frame processing rate from the HTTP request-response cycle, ensuring stable video streaming performance.

4.4 Subsystem Services

The backend services supporting the system include:

- Frame Streaming Service: A Django view function implementing HTTP multipart streaming (MJPEG format) that yields annotated frames from the shared frame queue, enabling browser-side live video display without WebSockets.
- Landmark Detection Service: The MediaPipe FaceLandmarker instance configured in VIDEO mode for temporal consistency, providing smoothed landmark tracks across consecutive frames.
- Metric Computation Service: Pure Python functions taking landmark arrays as input and returning scalar metric values (EAR, MAR, blink count), enabling unit testing without hardware dependencies.
- Alert Coordination Service: A singleton alert manager that maintains alert state, enforces inter-alert cooldown intervals, and interfaces with the pyttsx3 engine in a dedicated speech thread.
- Session Management Service: Django's built-in session framework extended with health session lifecycle hooks (session start, session end, summary generation).

4.5 User Interface Design

The web interface is structured as a single-page dashboard accessible via a browser at localhost. The primary view is divided into two panels: the left panel displays the live

MJPEG stream with facial landmark overlays and bounding box annotations; the right panel displays real-time numeric gauges for EAR, MAR, and Blink Rate, along with colour-coded wellness status indicators. A navigation sidebar provides access to the historical session records page, which renders a tabular summary of past sessions alongside trend charts rendered using Chart.js.

4.6 Summary

This chapter has described the modular design approach, Django MTV architecture, subsystem services, and user interface design of the Digital Wellness Monitoring System. The integrated design—combining MediaPipe's state-of-the-art landmark detection with a full Django web stack—represents a significant architectural novelty over existing standalone script-based fatigue detection tools.

CHAPTER-5

TECHNICAL IMPLEMENTATION AND ANALYSIS

5.1 Description

This chapter provides a detailed account of the technical implementation of the Digital Wellness Monitoring System, covering the Django project structure, the MediaPipe integration pipeline, the mathematical formulation of wellness metrics, the alert engine, and the results of functional and performance testing.

5.2 Technical Coding and Code Solutions

5.2.1 Django Project Structure

The repository is organized into two primary Django applications within the parent digital_wellness_system project: the monitor app (housing the core computer vision views, ORM models, and real-time streaming logic) and the smart_health app (handling user authentication, session management, and the historical analytics dashboard). A shared SQLite database (db.sqlite3) persists all session and metric data via Django's ORM. The face_landmarker.task binary file, downloaded from Google's MediaPipe model hub, resides at the project root and is loaded once at application startup.

5.2.2 MediaPipe Face Landmarker Integration

The FaceLandmarker is initialised using the Tasks API with the model asset path pointing to face_landmarker.task, configured for VIDEO running mode to enable temporal landmark smoothing across frames. Each OpenCV frame is converted from BGR to RGB colour space before being wrapped in a mediapipe.Image object and passed to the detect_for_video method along with the frame timestamp in milliseconds. The returned FaceLandmarkerResult object contains the 478 NormalizedLandmark instances (x and y normalised to frame width and height; z representing depth relative to the face mesh origin) used for all subsequent metric calculations.

5.2.3 Eye Aspect Ratio (EAR) Computation

For each eye, six landmark indices are selected from the Face Landmarker's 478-point topology: two horizontal landmarks defining the inner and outer eye corners, and four vertical landmarks defining the upper and lower eyelid boundaries. The EAR is computed as the average of the two vertical distances (p2-p6 and p3-p5) divided by the horizontal distance (p1-p4), following the formulation of Soukupova and Cech. The mean EAR across both eyes is computed for each frame. An EAR below the empirically determined threshold of 0.25 sustained for more than 20 consecutive frames triggers a drowsiness alert.

5.2.4 Mouth Aspect Ratio (MAR) Computation

The MAR is computed from four lip landmarks: the top of the upper lip, the bottom of the lower lip, and the two lip corners. The ratio of the vertical opening (top-to-bottom distance) to the horizontal width (corner-to-corner distance) is computed per frame. A MAR value exceeding the threshold of 0.6 for more than 15 consecutive frames is classified as a yawning event, triggering an appropriate audio alert.

5.2.5 Blink Rate Computation

The blink counter maintains a rolling state machine: when the mean EAR drops below the blink threshold (0.25) and subsequently rises above it within a short window, a blink event is registered. The blink rate (blinks per minute) is computed over a 60-second sliding window. A blink rate below 12 blinks per minute—approximately half the physiologically normal rate of 15-20 blinks/minute—triggers a visual strain alert encouraging the user to rest their eyes.

5.2.6 pyttsx3 Alert Engine

The pyttsx3 engine is initialised in a dedicated daemon thread to prevent the speech synthesis from blocking the main frame processing loop. An alert manager class maintains a cooldown dictionary keyed by alert type, ensuring that each alert type (drowsiness, yawning, eye strain) is not repeated within a configurable minimum interval (default 30 seconds). Alert messages are queued and consumed by the speech thread, enabling asynchronous, non-blocking audio feedback.

5.3 Algorithm

The system's operational algorithm is structured into the following phases:

- Phase 1 — Initialization: Load MediaPipe Face Landmarker model from face_landmarker.task. Initialize OpenCV VideoCapture on the default webcam. Instantiate metric state objects (EAR history buffer, blink counter, alert manager). Initialize pyttsx3 engine in a background thread. Start Django development server.

- Phase 2 — Real-Time Processing Loop: Read frame from OpenCV. Convert BGR to RGB. Pass RGB frame and timestamp to FaceLandmarker.detect_for_video(). Extract 478 NormalizedLandmark objects. Compute pixel coordinates by multiplying normalized values by frame dimensions. Compute EAR, MAR, and update blink counter. Compare metrics against thresholds. If threshold exceeded, enqueue alert message to TTS thread. Annotate frame with landmark dots, EAR/MAR values, and status text. Encode annotated frame as JPEG. Yield JPEG frame to multipart HTTP response stream.
- Phase 3 — Data Persistence: At session start, create a Django HealthSession ORM record. Every N frames (configurable), create a MetricSnapshot record with current EAR, MAR, and blink rate. At session end, update HealthSession with summary statistics.

5.4 Working Layout of Forms

The Django application presents the following user-facing pages:

- Login Form: Standard username/password authentication. Validates credentials against the Django auth backend and creates a session cookie.
- Monitoring Dashboard: The primary view. Displays the live MJPEG stream embedded in an <img> tag with a src pointing to the Django streaming URL. Metric values are updated via periodic AJAX calls to a JSON endpoint returning current EAR, MAR, and blink rate.
- Alert History View: A paginated table view of all alert events recorded in the current session, with timestamp, alert type, and metric value at time of alert.
- Session History View: A chart-based view of all past sessions, displaying EAR and blink rate trends over time using Chart.js rendered server-side via template variables.

5.5 Prototype Submission

A complete, functional prototype of the system is implemented as a Django project and is available in the GitHub repository at https://github.com/abhijeet10118/digital_wellness_system. The prototype demonstrates all key features: real-time webcam landmark detection, EAR/MAR/blink rate computation, audio alert generation, and health session data persistence. The system can be launched with a single 'python manage.py runserver' command after installing dependencies.

5.6 Testing and Validation

Functional testing was conducted with five student volunteers in a simulated study session environment. The face detection module successfully tracked landmarks across varying lighting conditions, glasses-wearing subjects, and head orientations within ±30 degrees. The EAR-based drowsiness detection achieved a precision of approximately 91% and recall of 87% against manually labelled ground truth drowsiness events. The MAR-based yawning detector correctly identified 19 of 22 yawn events across all test sessions. Audio alerts were consistently delivered within 2 seconds of threshold crossing, and no cross-session data contamination was observed in the SQLite persistence layer.

5.7 Performance Analysis

Processing performance was benchmarked on an Intel Core i5-10th Gen laptop with 8 GB RAM and the integrated graphics unit. The MediaPipe Face Landmarker achieved an average processing latency of 28ms per frame, corresponding to approximately 35 FPS, well above the 15 FPS minimum target. The Django streaming view introduced an average additional latency of 12ms per frame due to JPEG encoding and HTTP framing. Overall end-to-end latency from frame capture to browser display averaged 65ms, providing a practically imperceptible real-time experience.

Metric	Value
Average Frame Processing Latency (MediaPipe)	28 ms
Average HTTP Streaming Latency (Django)	12 ms
Average End-to-End Latency	65 ms
Effective Frame Rate	~35 FPS
EAR Drowsiness Detection Precision	~91%
EAR Drowsiness Detection Recall	~87%
MAR Yawn Detection Accuracy	86.4% (19/22 events)
Alert Delivery Latency	<2 seconds

5.8 Summary

This chapter has provided a comprehensive technical description of the implementation, covering Django structure, MediaPipe integration, metric algorithms, alert engine, and performance results. The system demonstrates real-time feasibility and strong detection accuracy, confirming the soundness of the architectural and algorithmic design choices.

CHAPTER-6

PROJECT OUTCOME AND APPLICABILITY

6.1 Outline

This chapter summarises the primary outcomes achieved by the Digital Wellness Monitoring System and analyses its applicability across a range of real-world domains.

6.2 Major Implementation Descriptions

The primary deliverables of this project are a fully functional Django web application with integrated real-time MediaPipe facial monitoring, a clinically grounded wellness analytics engine computing EAR, MAR, and blink rate from 478 3D facial landmarks, a non-intrusive pyttsx3-powered audio alert system with configurable thresholds and cooldown management, and a persistent SQLite-backed health session record system accessible via a browser-based dashboard.

6.3 Main Project Successes

The key achievements of this project are as follows:

- Successful real-time integration of MediaPipe Face Landmarker (478 3D landmarks) within a Django MTV web application—a novel combination not present in existing academic implementations.
- Demonstration of effective drowsiness and yawning detection using EAR and MAR metrics computed entirely from landmark geometry, with no requirement for specialized hardware.
- Design and implementation of a privacy-preserving system where no video or biometric data leaves the user's device, addressing a critical gap in existing cloud-based wellness solutions.

- Achievement of greater than 35 FPS processing throughput on consumer-grade hardware, making the system practically deployable without dedicated GPU infrastructure.
- Development of a persistent health records module enabling longitudinal wellness trend analysis, a feature absent from all comparable open-source fatigue detection tools.

6.4 Real-World Applicability

The system has significant potential across multiple domains:

- Student Wellness in Educational Institutions: Deployed on student workstations in computer labs or study rooms, the system monitors fatigue levels during extended study sessions, alerting students to take breaks and preventing the accumulation of digital eye strain and cognitive fatigue.
- Workplace Ergonomics and Occupational Health: Corporate wellness programs can deploy the system on employee workstations, providing HR departments with aggregated (anonymised) wellness data to design evidence-based intervention policies.
- Driver Safety and Fleet Monitoring: With minor adaptation, the EAR-based drowsiness detection module can be embedded in an in-vehicle system to alert drowsy drivers, directly addressing a leading cause of road accidents.
- Online Learning and E-Examination Platforms: Education technology platforms can integrate the monitoring module to detect disengagement or fatigue in remote learners, enabling adaptive content pacing.
- Clinical and Research Applications: Researchers studying fatigue, attention, and cognitive load can use the system's persistent session logs as an objective, non-invasive data collection tool.

6.5 Inference

The project outcomes confirm that affordable, privacy-preserving, real-time digital wellness monitoring is technically achievable using widely available consumer hardware and open-source software. By integrating MediaPipe, OpenCV, Django, and pyttsx3 into a coherent platform, this project demonstrates a replicable blueprint for proactive health monitoring that can meaningfully improve user well-being across educational, professional, and clinical contexts.

CHAPTER-7

CONCLUSIONS AND RECOMMENDATIONS

7.1 Outline

This final chapter synthesises the project's findings, acknowledges its current limitations, proposes directions for future development, and documents the individual contributions of each team member.

7.2 Limitations and Constraints

Despite its achievements, the system has several limitations that should be acknowledged:

- Threshold Generalizability: EAR and MAR thresholds are empirically determined and may require individual calibration, as facial geometry varies significantly across individuals, ethnicities, and ages.
- Single-Face Constraint: The current implementation monitors a single face at a time, limiting deployment to individual workstations rather than classroom-wide monitoring.
- Occlusion Sensitivity: Accessories such as eyeglasses with thick frames or face masks can partially occlude the landmark detection region, reducing metric accuracy.

- Internet Dependency for Initial Setup: While the runtime is fully local, the initial download of the MediaPipe model file and Python dependencies requires internet access.
- No Long-Term Trend Intelligence: The current analytics layer presents raw historical data; no machine learning model has yet been trained to predict wellness trajectories or recommend interventions.

7.3 Future Enhancements

Several promising avenues exist for future development:

- Multi-User Monitoring: Extend the system to simultaneously track multiple users in a shared environment using MediaPipe's multi-face detection capability.
- Personalized Threshold Calibration: Implement a brief calibration protocol at session start to adapt EAR and MAR thresholds to each individual user's baseline facial geometry.
- ML-Based Wellness Prediction: Train a time-series classification model (e.g., LSTM or Transformer) on accumulated session data to predict approaching fatigue episodes before they manifest, enabling proactive rather than reactive interventions.
- Wearable Integration: Integrate heart rate and skin conductance data from consumer wearables (e.g., Fitbit, Garmin) to complement facial metrics with physiological signals, producing a richer composite wellness score.
- Cloud Deployment with Privacy Preservation: Explore federated learning or on-device model fine-tuning architectures that allow personalization without transmitting raw biometric data to a central server.
- Mobile Application: Port the monitoring module to a mobile platform using MediaPipe's Android/iOS libraries, expanding accessibility to tablet and smartphone-based study environments.

7.4 Inference

This project has successfully demonstrated that a real-time, AI-powered digital wellness monitoring system—offering drowsiness detection, yawning analysis, blink rate tracking, audio alerts, and persistent health records—can be implemented as a fully functional Django web application deployable on consumer hardware. The integration of MediaPipe Face Landmarker with a full Django web stack constitutes a technically novel contribution to the growing field of AI-assisted human wellness monitoring. While limitations exist in the current implementation, the clear roadmap for future enhancements positions this project as a strong foundation for a commercial-grade digital wellness platform.

7.5 Contributions

24MIM10074 — Abhijeet Sandilya (Project Lead)

As project leader, Abhijeet was responsible for the overall system architecture and the integration of all subsystems into a coherent Django application. He designed the Django project structure (splitting concerns across the monitor and smart_health apps), established the GitHub repository and version control workflow, and implemented the core MediaPipe Face Landmarker integration pipeline. His primary technical contribution was the development of the frame streaming view and the shared-memory threading architecture that decouples the computer vision pipeline from the HTTP request-response cycle. The computer vision metric development, implementing the mathematical formulations for EAR, MAR, and blink rate computation from MediaPipe's 478-landmark coordinate output. He conducted threshold calibration experiments across the five test volunteers, establishing the default EAR, MAR, and blink rate limits used in the production system. He also developed the unit test suite for the metric computation module.

24MIM10221 — Ayush Chandravanshi

Ashish was responsible for the pyttsx3 alert engine implementation. He designed the multi-threaded alert architecture—with a dedicated speech synthesis daemon thread and an inter-alert cooldown management system—that prevents alert flooding while ensuring timely delivery. He also implemented the alert event logging ORM model and the corresponding Django admin interface for reviewing alert histories.

24MIM10038 — Atharva Padole

Atharva developed the Django ORM models (HealthSession, MetricSnapshot), managed all database migrations, and implemented the smart_health app's session history dashboard. He integrated Chart.js into the Django template layer to render EAR and blink rate trend visualisations from server-side session data. He also conducted the functional testing sessions with student volunteers and compiled the performance benchmark results , And designed the pipeline for RAG Chatbot.

24MIM10098 — Soma Vashista

Soma was responsible for the full front-end HTML/CSS implementation of the Django templates, including the login page, monitoring dashboard, alert history view, and session history chart page. She ensured cross-browser compatibility (Chrome, Firefox, Edge) and responsive layout. She also performed comprehensive system testing across Windows 10, Windows 11, and Ubuntu 20.04, documenting platform-specific setup instructions for the user manual.

APPENDIX A: TECHNICAL DETAILS

1. Model and Dataset Details

1.1 MediaPipe Face Landmarker Model

The primary AI model used in this project is the MediaPipe Face Landmarker, distributed as the face_landmarker.task bundle. This model detects 478 three-dimensional facial landmarks, including the standard 468 Face Mesh landmarks plus 10 additional iris landmarks. It is built on top of BlazeFace for face detection and a MobileNet-V2-based landmark regression network. The model operates in VIDEO mode for this project, enabling inter-frame landmark smoothing via a Kalman filter-like temporal tracker. The model file is approximately 7.8 MB and is loaded once at Django application startup via the mediapipe.tasks.vision.FaceLandmarker.create_from_options API.

1.2 EAR Landmark Indices

The following MediaPipe Face Landmarker landmark indices are used for EAR computation (mapped from the 478-point topology):

Eye	Landmark Role	Index (approx.)
Left Eye	Outer corner (p1)	263
Left Eye	Upper-outer (p2)	387
Left Eye	Upper-inner (p3)	385
Left Eye	Inner corner (p4)	362
Left Eye	Lower-inner (p5)	380
Left Eye	Lower-outer (p6)	373
Right Eye	Outer corner (p1)	33
Right Eye	Upper-outer (p2)	160
Right Eye	Upper-inner (p3)	158
Right Eye	Inner corner (p4)	133
Right Eye	Lower-inner (p5)	153
Right Eye	Lower-outer (p6)	144

2. Django Application Module Details

2.1 monitor App

The monitor Django app contains the real-time streaming view (views.py), OpenCV VideoCapture management, the MediaPipe integration class, and the metric computation functions. The primary model in this app is MetricSnapshot, which records EAR, MAR, blink count, and alert_triggered fields per snapshot event. URL routes expose the streaming endpoint (/monitor/stream/) and the JSON metrics endpoint (/monitor/metrics/json/).

2.2 smart_health App

The smart_health app manages HealthSession lifecycle, user authentication views, and the historical dashboard. The HealthSession model records session_start, session_end, avg_ear, avg_mar, total_blinks, total_alerts, and a foreign key to the Django User model. The dashboard view queries the last 10 sessions for the authenticated user and passes summary statistics to the template for Chart.js rendering.

2.3 health_monitor_db

The health_monitor_db directory contains Django migration files for both apps and the SQLite database schema. Database initialization is performed via Django's standard 'python manage.py migrate' command, which creates all required tables using the ORM model definitions.

APPENDIX B: USER MANUAL AND SETUP INSTRUCTIONS

1. Prerequisites

Before installation, ensure the following are available on your system:

- Python 3.8 or later with pip installed.
- A working webcam (built-in or USB, 720p or higher recommended).
- A modern web browser (Chrome, Firefox, or Edge).
- Git (optional, for cloning the repository).

2. Installation

Step 1: Clone or download the project repository.

1. git clone https://github.com/abhijeet10118/digital_wellness_system.git

Step 2: Navigate to the project directory and install all Python dependencies:

2. pip install -r requirements.txt

Note: On some systems, pyttsx3 may require additional system-level TTS engine installation. On Linux, install espeak: sudo apt-get install espeak. On Windows, the built-in SAPI5 engine is used automatically.

Step 3: Download the MediaPipe Face Landmarker model file and place it at the project root as face_landmarker.task. The file can be obtained from the Google MediaPipe model hub.

3. Database Initialization

Run the following commands to initialize the SQLite database:

3. python manage.py makemigrations
4. python manage.py migrate
5. python manage.py createsuperuser

4. Running the Application

Launch the Django development server with:

6. python manage.py runserver

Open a web browser and navigate to http://127.0.0.1:8000. Log in with the superuser credentials created in the previous step.

5. Using the Dashboard

- Upon login, the monitoring dashboard is displayed. Click 'Start Monitoring' to activate the webcam and begin real-time wellness tracking.

- Ensure your face is clearly visible within the webcam frame and well-lit. The system will display real-time EAR, MAR, and blink rate values alongside the annotated video feed.
- Audio alerts will be spoken through your default audio output device when wellness thresholds are breached.
- Navigate to 'Session History' in the sidebar to review past sessions and wellness trends.
- Click 'Stop Monitoring' to end the session and save the summary record to the database.

REFERENCES

I. Research Papers

1. Soukupova, T., & Cech, J. (2016). Real-time eye blink detection using facial landmarks. In 21st Computer Vision Winter Workshop. (Introduces the Eye Aspect Ratio metric that forms the basis of the drowsiness detection module.)
2. Kazemi, V., & Sullivan, J. (2014). One millisecond face alignment with an ensemble of regression trees. In CVPR 2014. IEEE. (Established real-time facial

landmark regression using ensemble methods, predecessor to modern landmark detectors.)

3. Lugaresi, C., Tang, J., Nash, H., et al. (2019). MediaPipe: A framework for perceiving and processing reality. In Third Workshop on Computer Vision for AR/VR at IEEE CVPR 2019. (Introduces the MediaPipe framework underlying the Face Landmarker model used in this project.)

4. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. (Foundational reference for the OpenCV library used for webcam capture and frame processing.)

5. Harris, C.R., Millman, K.J., van der Walt, S.J., et al. (2020). Array programming with NumPy. Nature, 585, 357-362. (Reference for NumPy, used for all numerical landmark coordinate computations including EAR and MAR.)

6. Maggioni, A., Ruf, M., Huber, R., Musch, A., Siclari, F., & Bassetti, C.L. (2021). Eye tracking as a measure of drowsiness in long-distance truck drivers. Journal of Sleep Research, 30(3). (Validates EAR-based drowsiness detection in real-world fatigue monitoring contexts.)

7. Abtahi, S., Omidyeganeh, M., Shirmohammadi, S., & Hariri, B. (2014). YawDD: A yawning detection dataset with application to drowsy drivers. In MMSys 2014. (Validates MAR-based yawning detection, relevant to the MAR module of this project.)

8. Django Software Foundation. (2023). Django Documentation, Version 4.2. https://docs.djangoproject.com (Official reference for the Django web framework used as the application backbone.)

II. Books

9. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. (Foundational text covering CNNs and deep learning architectures underlying MediaPipe's landmark models.)

10. Szeliski, R. (2022). Computer Vision: Algorithms and Applications (2nd ed.). Springer. (Comprehensive reference for computer vision algorithms including face detection and landmark tracking.)

11. Holovaty, A., & Kaplan-Moss, J. (2009). The Definitive Guide to Django (2nd ed.). Apress. (Reference text for Django architectural patterns including MTV, ORM, and URL routing.)

III. Websites and Online Resources

12. Google AI Edge. (2024). MediaPipe Face Landmarker. https://developers.google.com/mediapipe/solutions/vision/face_landmarker (Official documentation for the MediaPipe Face Landmarker model and Tasks API used in this project.)

13. Django Project. (2024). Django Documentation. https://docs.djangoproject.com (Official Django framework reference for MTV architecture, ORM, views, and templates.)

14. thevickypedia. (2024). pyttsx3 Documentation. https://pyttsx3.readthedocs.io (Documentation for the pyttsx3 offline TTS library used for audio wellness alerts.)

15. OpenCV Team. (2024). OpenCV Documentation. https://docs.opencv.org (Reference for VideoCapture, image color conversion, and JPEG encoding functions.)

16. Kaggle. (2016). Facial Expression Recognition Challenge (FER-2013). https://www.kaggle.com/c/challenges-in-representation-learning-facial-expression-recognition-challenge (Community-standard benchmark dataset providing context for facial metric validation.)